



Educación
Secretaría de Educación Pública



TECNOLÓGICO
NACIONAL DE MÉXICO®



TECNOLÓGICO NACIONAL DE MÉXICO INSTITUTO TECNOLÓGICO DE LEÓN

Ingeniería en Sistemas Computacionales

Sistemas Programables

Integración Total MQTT (ESP32 ↔ Python)

PRESENTA:

Rivera Ponce David Eduardo
Ramirez Abundiz Berenice
Varela Ambriz Saúl



Contenido

Introducción	3
Objetivo General	4
Objetivos Específicos.....	4
ARQUITECTURA GENERAL DEL SISTEMA.....	5
.....	5
ESTRUCTURA DEL REPOSITORIO	6
Matriz de Tópicos	7
Anexos de código	10
Conclusiones	14
Link al repo:	16



Introducción

En la actualidad, la tecnología IoT permite desarrollar sistemas inteligentes capaces de ayudar a las personas mediante sensores, comunicación inalámbrica y procesamiento de datos en tiempo real. Gracias a esto, es posible crear dispositivos que mejoren la seguridad, accesibilidad y autonomía de los usuarios en diferentes situaciones cotidianas.

El presente proyecto consiste en el desarrollo de un Bastón Inteligente, diseñado para apoyar al usuario mediante la detección de obstáculos, movimiento y localización remota. El sistema utiliza una ESP32 como unidad principal, sensores electrónicos y comunicación MQTT para enviar y recibir información en tiempo real.

Además, el proyecto integra un sistema de inteligencia artificial utilizando OpenCV y una ESP32-CAM, permitiendo detectar personas mediante procesamiento de imágenes. Cuando el sistema identifica una persona o una situación importante, se activan actuadores como el motor de vibración, la bocina o el zumbador para alertar al usuario.

Para mejorar la organización y mantenimiento del software, se implementó una arquitectura basada en HAL (Hardware Abstraction Layer), separando la lógica del sistema del acceso directo al hardware. También se utilizó el protocolo MQTT para lograr una comunicación eficiente entre los dispositivos y el servidor.

Con este proyecto se busca demostrar cómo la integración de IoT, sensores, MQTT e inteligencia artificial puede aplicarse en un sistema funcional y útil, capaz de responder de forma inteligente a diferentes situaciones del entorno.



Objetivo General

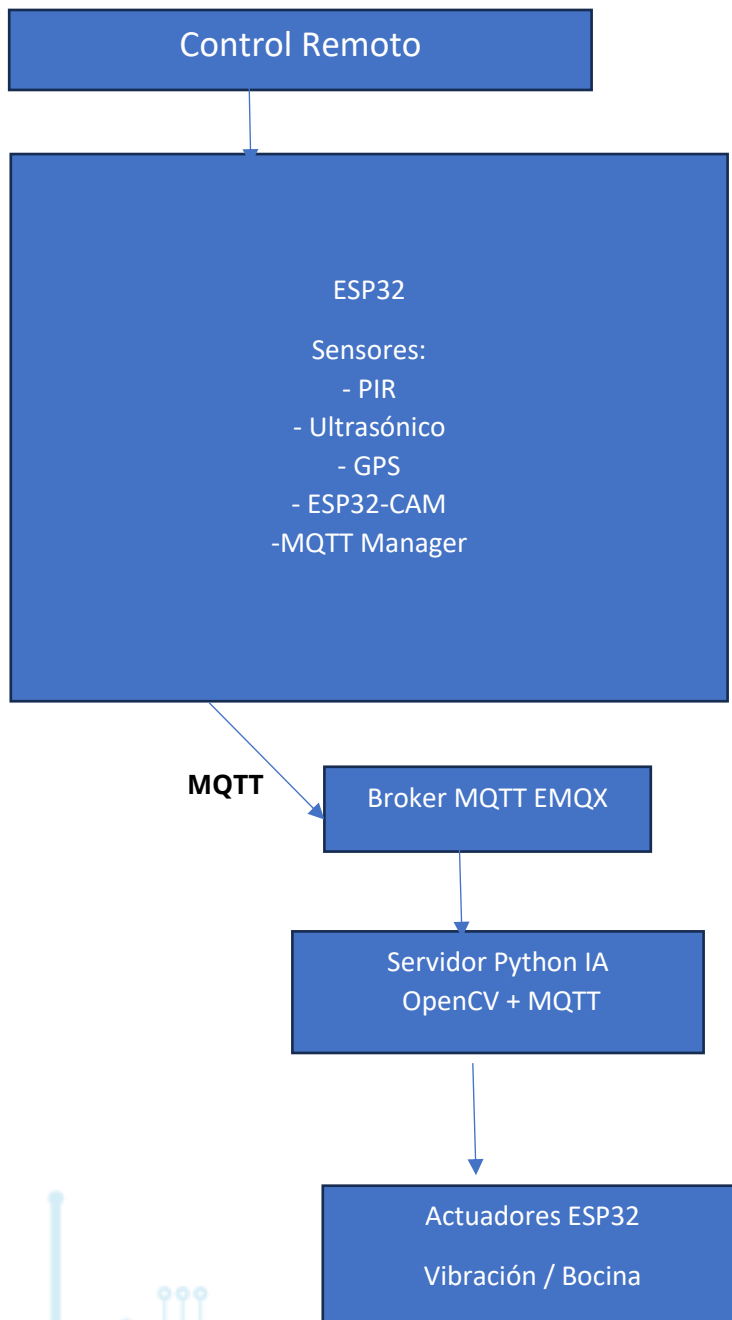
Diseñar e implementar un sistema IoT basado en ESP32 y MQTT capaz de monitorear sensores, procesar información en tiempo real y activar actuadores de manera remota mediante una arquitectura modular basada en HAL e integración de inteligencia artificial.

Objetivos Específicos

- Implementar comunicación inalámbrica mediante el protocolo MQTT para el envío y recepción de datos en tiempo real.
- Integrar sensores como ultrasónico, PIR, GPS y ESP32-CAM para la obtención de información del entorno.
- Implementar actuadores como motor de vibración, bocina y zumbador para generar alertas y respuestas físicas.
- Aplicar una arquitectura HAL (Hardware Abstraction Layer) para separar la lógica del sistema del acceso directo al hardware.
- Implementar mecanismos de reconexión automática para mejorar la estabilidad de la comunicación MQTT y WiFi.
- Integrar un sistema de inteligencia artificial utilizando OpenCV y ESP32-CAM para la detección de personas mediante procesamiento de imágenes.

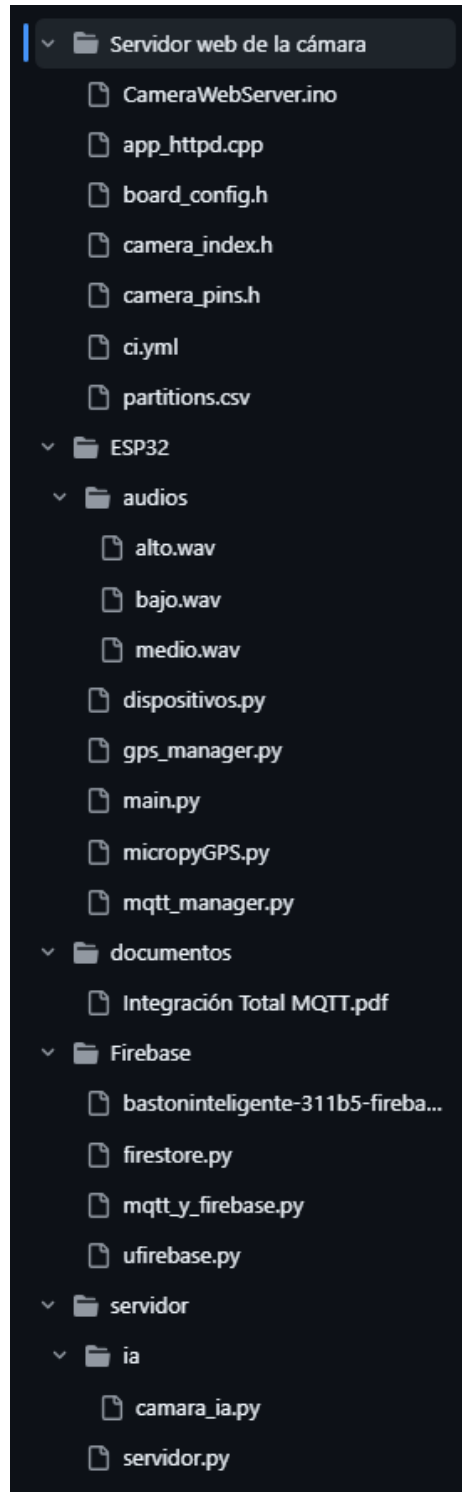


ARQUITECTURA GENERAL DEL SISTEMA





ESTRUCTURA DEL REPOSITORIO





Matriz de Tópicos

ID	Topico_MQTT	Nivel_1_Proyecto	Nivel_2_Tipo	Nivel_3_Modulo	Nivel_4_ID	Dirección	Descripción	Tipo_Dato
1	bastoninteligente/sensor/ultrasonico/01	bastoninteligente	sensor	ultrasonico	01	ESP32 -> Servidor	Publica la distancia detectada por el sensor HC-SR04 en centímetros	int
2	bastoninteligente/sensor/pir/01	bastoninteligente	sensor	pir	01	ESP32 -> Servidor	Publica la detección de movimiento del sensor PIR	bool
3	bastoninteligente/sensor/control/01	bastoninteligente	sensor	control	01	ESP32 -> Servidor	Publica el botón presionado del control remoto	string
4	bastoninteligente/sensor/gps_latitud/01	bastoninteligente	sensor	gps_latitud	01	ESP32 -> Servidor	Publica la latitud obtenida desde el GPS	float



5	bastoninteligente/sensor/gps_longitud/01	bastoninteligente	sensor	gps_longitud	01	ESP32 -> Servidor	Publica la longitud obtenida desde el GPS	float
6	bastoninteligente/sensor/camara/01	bastoninteligente	sensor	camara	01	ESP32-CAM -> Servidor	Publica eventos relacionados con captura de imagen	string
7	bastoninteligente/comando/zumbador/01	bastoninteligente	comando	zumbador	01	Servidor -> ESP32	Envío de comando para activar el zumbador	string
8	bastoninteligente/comando/vibracion/01	bastoninteligente	comando	vibracion	01	Servidor -> ESP32	Envío de comando para activar el motor vibrador	string
9	bastoninteligente/comando/bocina/01	bastoninteligente	comando	bocina	01	Servidor -> ESP32	Envío de comando para reprod	string



							ucir audio	
1 0	bastoninteligente/coma ndo/captura/01	bastonint eligente	coman do	captura	01	Servi dor - > ESP3 2- CAM	Coma ndo para captur ar fotogr afia	string
1 1	bastoninteligente/actua dor/zumbador/01	bastonint eligente	actuad or	zumbador	01	ESP3 2 -> Servi dor	Estado operat ivo actual del zumba dor	string
1 2	bastoninteligente/actua dor/vibracion/01	bastonint eligente	actuad or	vibracion	01	ESP3 2 -> Servi dor	Estado operat ivo actual del motor vibrad or	string
1 3	bastoninteligente/actua dor/bocina/01	bastonint eligente	actuad or	bocina	01	ESP3 2 -> Servi dor	Estado operat ivo actual de la bocina	string
1 4	bastoninteligente/ia/pe rsona_detectada/01	bastonint eligente	ia	persona_d etectada	01	Servi dor IA -> ESP3 2	Result ado de detecc ion de person a media nte OpenC V	bool
1 5	bastoninteligente/fireb ase/foto/01	bastonint eligente	firebas e	foto	01	Servi dor IA ->	Registr o de image n	string



						Firebase	capturada por la ESP32-CAM	
--	--	--	--	--	--	----------	----------------------------	--

Anexos de código

```
while True:
    try:
        # 1 -- REVISAR MENSAJES MQTT
        servidor.escuchar()

        # 2 -- PUBLICAR TELEMETRÍA
        ahora = time.time()

        # Publicar sensores cada 5 segundos
        if (ahora - ultima_publicacion) >= 5:
            servidor.publicar_sensores()
            ultima_publicacion = ahora

        # ENCARGA DE PUBLICAR Y OBTENER DATOS DE FIREBASE CADA 20 segundos
        if (ahora - ultima_firebase) >= 20:
            try:
                servidor.enviar_sensores_firebase() #enviamos los datos a firebase
                #Leemos los datos de firebase con ello tambien podemos tomar desiciones al igual que el control remoto
                servidor.leer_actuadores_firebase()
                ultima_firebase = ahora
            except Exception as e:
                print("Error Firebase:", e)

        # 3 -- Lógica del funcionamiento del bastón:
        control_valor = sensores.control_obtener_valor()

        if control_valor == "ultrasonico_leer_cm":
            print("\nActivando ultrasonico y bocina...")
            servidor.bocina_publicar_encendido()

        elif control_valor == "pir_movimiento":
            pir_valor = sensores.pir_obtener_valor()

            print("\nValor del PIR...")
            print(pir_valor)
            # Falta la implementación de la cámara cuando 'pir_valor = 1'
            # y el motor de vibración
            #
            #
            if pir_valor == 1:
                servidor.vibracion_publicar_encendido()

        elif control_valor == "zumbador_localizador":
            print("\nActivando Zumbador...")
            servidor.zumbador_publicar_encendido()

        time.sleep(0.1)
    except Exception as e:
        print("=====")
        print("ERROR EN LOOP PRINCIPAL")
        print(e)
        print("=====")

        # Intento de reconexión MQTT
        servidor.reconectar()

        time.sleep(2)
```

Algoritmo de reconexión automática implementado en el loop principal del sistema.

Este bloque permite detectar excepciones durante la ejecución del sistema MQTT y restablecer automáticamente la conexión con el broker en caso de pérdida de red, caída del servidor o error de comunicación. La implementación mediante try-except mejora la tolerancia a fallos del Bastón Inteligente y evita reinicios manuales del dispositivo.



```
# Instancia de sensores
sensores = SensorBox()

# Instancia de actuadores
actuadores = ActuatorBox()

print("HAL inicializada correctamente")

# INICIALIZACIÓN MQTT
print("Iniciando MQTT + Firebase...")

servidor = Servidor(
    sensores,
    actuadores
)

# CONEXIÓN WIFI
servidor.conectar_wifi()

# CONEXIÓN MQTT
servidor.conectar_mqtt()

print("Sistema MQTT iniciado correctamente")
```

Integración desacoplada mediante arquitectura HAL.

La lógica principal no accede directamente al hardware físico. Todas las acciones sobre sensores y actuadores son ejecutadas mediante clases encapsuladas (SensorBox y ActuatorBox), garantizando modularidad, mantenimiento y seguridad del sistema.



```
# Callback principal MQTT
self.cliente.set_callback(
    self.mensaje_recibido
)
```

Configuración del callback principal MQTT.

El sistema implementa un callback MQTT que permite ejecutar automáticamente una función cuando se recibe un mensaje desde el broker. Esto facilita la recepción en tiempo real de comandos remotos para actuadores del Bastón Inteligente.



```
# PUBLICAR SENSORES
```

```
def publicar_sensores(self):
```

```
    """
```

```
    Publica la telemetría de todos los sensores  
    del Bastón Inteligente hacia el broker MQTT.  
    """
```

```
# Obtención centralizada de sensores
```

```
datos = self.sensores.obtener_resumen_sensores()
```

```
# ULTRASÓNICO
```

```
self.cliente.publish(  
    b"baston_equipo_7718/sensores/ultrasonico",  
    str(datos["ultrasonico"])  
)
```

```
# PIR
```

```
self.cliente.publish(  
    b"baston_equipo_7718/sensores/pir",  
    str(datos["pir"])  
)
```

```
# CONTROL IR
```

```
self.cliente.publish(  
    b"baston_equipo_7718/sensores/control",  
    str(datos["control"])  
)
```

Publicación de telemetría de sensores mediante MQTT.

El sistema publica continuamente información proveniente de los sensores ultrasónico, PIR y control remoto hacia el broker MQTT, permitiendo monitoreo remoto y procesamiento externo en tiempo real.

Suscripción MQTT para recepción de comandos remotos.

La ESP32 permanece suscrita a tópicos MQTT específicos para recibir instrucciones remotas relacionadas con los actuadores del sistema, permitiendo control distribuido y desacoplado.



Conclusiones

David:

Problema: El reto principal fue que ya tenía la HAL construida de la tarea anterior, pero al intentar conectarla con MQTT, me di cuenta de que algunos sensores, como el receptor IR y el ultrasónico, enviaban datos tan rápido que saturaban el script de Python. Al principio, los mensajes se amontonaban y los actuadores (como el zumbador y el motor de vibración) tardaban varios segundos en responder a los comandos que enviaba desde la computadora.

Solución: En lugar de rehacer todo, adapté las funciones que ya tenía en mi HAL para que trabajaran en conjunto con el cliente MQTT. Configuré el código para que los sensores solo publicaran información cuando hubiera un cambio real (por ejemplo, cuando el PIR detectara movimiento) y no todo el tiempo. Esto limpió mucho la comunicación y permitió que la ESP32 pudiera escuchar los comandos de los actuadores sin retrasos, aprovechando la estructura que ya había programado antes.

Conclusión: Haber hecho la HAL previamente fue una gran ventaja, ya que solo tuve que 'puentear' mis funciones de lectura y escritura con los tópicos de MQTT. Me quedó claro que si el hardware está bien organizado desde el inicio, conectarlo a internet es mucho más sencillo. Logré que todo el sistema funcionara de forma estable y aprendí a gestionar mejor el flujo de datos para no saturar el servicio.

Saul:

Problema: Durante la integración del protocolo MQTT con la ESP32 surgieron dificultades relacionadas con la comunicación simultánea entre sensores y actuadores. En las primeras pruebas, la publicación constante de datos generaba demasiados mensajes en el broker, provocando retrasos en la recepción de comandos y una respuesta inestable de algunos actuadores. Además, fue necesario adaptar la lógica existente del proyecto para mantener el desacoplamiento entre la comunicación MQTT y el acceso al hardware.

Solución: Para resolver el problema se aprovechó la arquitectura HAL desarrollada previamente, utilizando sus funciones como intermediarias entre el hardware y el módulo MQTT. Se reorganizó el flujo de telemetría para enviar únicamente eventos relevantes y lecturas importantes, reduciendo la saturación de mensajes. También se separó la lógica de comunicación en un módulo independiente (`mqtt_manager.py`), permitiendo que la ESP32 pudiera administrar correctamente tanto la publicación de sensores como la recepción de comandos remotos sin afectar el funcionamiento general del sistema.

Conclusión: La implementación de MQTT permitió establecer una comunicación estable entre la ESP32 y el servidor Python, facilitando el monitoreo y control remoto del Bastón Inteligente. La utilización de la HAL simplificó la integración y evitó modificaciones directas sobre el hardware,



haciendo el sistema más organizado, escalable y fácil de mantener. Gracias a esta práctica se comprendió la importancia de modularizar el software embebido y de optimizar el flujo de datos en aplicaciones IoT para lograr un comportamiento más eficiente y confiable.

Berenice:

Problema: Uno de los principales inconvenientes durante el desarrollo fue lograr que la comunicación MQTT funcionara correctamente sin afectar el rendimiento de la ESP32. En varias pruebas iniciales, el sistema presentaba comportamientos inestables debido a que algunos sensores realizaban lecturas demasiado frecuentes, ocasionando tráfico innecesario en el broker. Esto provocaba pérdidas de sincronización entre la recepción de datos y la ejecución de comandos enviados desde el servidor Python, especialmente cuando varios componentes trabajaban al mismo tiempo.

Solución: Para mejorar el funcionamiento del sistema se reorganizó la estructura del proyecto utilizando una arquitectura modular basada en HAL. De esta manera, la comunicación MQTT quedó separada del acceso directo al hardware, permitiendo que cada componente tuviera una función específica dentro del sistema. También se optimizó la publicación de datos, limitando el envío de información únicamente a eventos importantes o lecturas periódicas controladas, reduciendo considerablemente la carga de mensajes y mejorando la estabilidad de la conexión entre la ESP32 y el broker MQTT.

Conclusión: La integración total mediante MQTT permitió convertir el Bastón Inteligente en un sistema IoT mucho más flexible y escalable. Gracias a la separación entre la HAL y el módulo de comunicación, fue posible mantener un código más limpio, reutilizable y sencillo de mantener. Además, esta práctica ayudó a comprender la importancia de diseñar arquitecturas desacopladas en proyectos embebidos, ya que facilitan tanto la integración de nuevas tecnologías como la detección y solución de errores durante el desarrollo.



Educación
Secretaría de Educación Pública



TECNOLÓGICO
NACIONAL DE MÉXICO®



Link al repo:

Link del Repositorio en Github:

<https://github.com/SaulVarelaAmbriz/Baston-Inteligente-MQTT.git>